Large Scale Optimization for Transport & Mobility

Assignment 1

Deadline: Friday September 18, 17:00

Rolf van Lieshout

1. This is an individual assignment. Therefore, it is **not** allowed to collaborate with others (helping each other with small coding questions is allowed). Copying code or text from fellow students is considered plagiarism.

2. AI use in this assignment is **regulated per step**: the implementations in steps 3–5 must be written by yourself, without AI assistance beyond debugging code you have already written. In step 6 you are explicitly required to use an AI assistant and to document its output. For the report, you may ask AI for suggestions to improve your writing, but copy-pasting full sentences is not allowed. Don't lose your voice!

3. Test your implementations on small instances you constructed yourself, and verify that the results are correct. It is often good practice to write a little bit of verification code to verify the correctness of your solutions.

GRASPing Expensive Insertion

In this assignment, you will develop and implement a Greedy Randomized Adaptive Search Procedure (GRASP) based on *expensive insertion* to solve the Traveling Salesman Problem (TSP). After selecting an initial city, the algorithm creates a tour by joining it together to city with the longest distance from the initial city. Then, in each iteration, among all cities not in the tour, we choose the city whose minimum distance to a city in the tour is the largest (i.e., we select the city that is as far from all in-tour cities as possible). Finally, we insert the selected city in the position that causes the smallest increase in tour length. Figure 1 illustrates this expensive insertion heuristic for a small TSP instance.

To get you started, we have prepared code for the structure and basic functionalities of the program, which you can download via Canvas. The *zip-file* contains the following files:

- **`TSP.py`:** the Python file that contains a class that represents a Traveling Salesman Problem. The class already has methods for reading TSP instances, evaluating solutions and applying the nearest neighbour heuristic. You will have to extend this class.

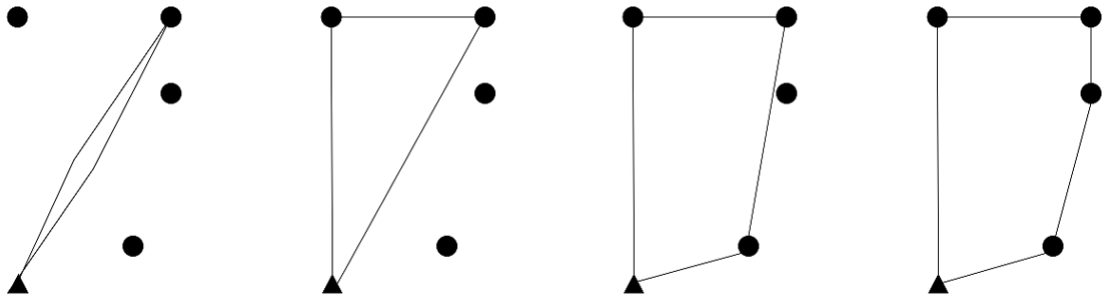


Figure 1: An example of running the expensive insertion on a small TSP instance

- The folder "Instances": a folder with a large set of benchmark instances of different sizes.

You are asked to perform the following steps:

1. Make a selection of ten instances that you will use to test your code: five small instances, three medium instances and two large instances. Set a random seed to ensure that your results are reproducible.

2. Run the nearest neighbour heuristic for different starting positions. Comment on the variation in solution quality.

3. Write a method `getTour_ExpensiveInsertion` that executes the expensive insertion as described above. (1 point)

4. There are two steps in the expensive insertion that can be randomized. Identify these two steps, and develop a GRASP algorithm by randomizing one of them, or both. Implement your algorithm in a method within the TSP class called `getTour_GRASPedInsertion`. (1 point)

5. Write a method in the TSP class called `isTwoOpt` that takes a tour as input and checks whether it is 2-optimal. Next, write a method called `makeTwoOpt` that takes in a tour as input and applies 2-exchanges until the tour is 2-optimal (i.e. `isTwoOpt` returns `True`). In your methodology section, state and motivate the design choices you made: first-improvement or best-improvement, the order in which you scan candidate pairs etc. (1 point)

6. Only after completing step 5, open a new conversation with an AI assistant of your choice and give it exactly the prompt stated in Appendix A of this assignment, with no additional context. Add the resulting method to your TSP class as `makeTwoOpt_AI`, unmodified except for the minimal changes needed to make it run; you may prompt further for corrections, but every round must be logged. Compare the two implementations on your ten instances, starting from identical initial tours and using identical seeds, on:

    - tour quality: are the resulting tour lengths equal, and does `isTwoOpt` accept both outputs?
    - computation time

- design choices: what did the generated code do differently, and which of these differences explains the observed runtime gap?

Include in your appendix the full conversation transcript, the number of correction rounds, every change you made to the generated code, and the name and version of the AI model you used. (2 points)

7. Develop an algorithm that combines the GRASPed insertion heuristic with 2-opt and implement this algorithm in the TSP class. Make a figure similar to the one on Slide 26 of the lecture slides on Local Search, with the costs before local search on the x-axis, and after local search on the y-axis. (1 point)

Submission Requirements

Summarize your findings in a report of at most 6 pages (font size = 11pt, margin = 1 inch). Your report should include

1. a short introduction that states and motivates the problem and research (i.e., explains its relevance), summarizes the solution approach and the main results, ($\frac{1}{2}$ point)
2. a methodology section that outlines the developed algorithms, motivates the design choices (e.g. efficiency considerations or how you select the pair of arcs to be swapped in a 2-opt iteration), and discusses how you validated the correctness of your implementation, ($1\frac{1}{2}$ points)
3. a results section that presents and discusses the results of the computational experiments that you conducted, ($1\frac{1}{2}$ points)
4. a short conclusion that summarizes your research and discusses its limitations and how your algorithms/implementation could be improved, ($\frac{1}{2}$ point)
5. and an appendix that contains your Python code and the documentation required in step 6 (the appendix does not count towards the 6-page limit).

Your submission should contain both the report in pdf format, and a zip-file that contains your code and the ten instances that you used to test your algorithms. Your code must be submitted as plain Python (`.py`) files; Jupyter notebooks (`.ipynb`) are not accepted.

Assessment

This assignment will be assessed based on solution correctness and efficiency (0-6 points) and on the quality of the report (0-4 points). The sum of the obtained points directly translates to your grade. The comparison in step 6 is assessed on the rigour of the experimental setup and on how well you explain the differences between the two implementations. Concluding that the generated code outperforms your own is an acceptable and fully creditable outcome, provided the comparison is sound.

In case anything about the assignment is unclear, please use the designated discussion board on Canvas.

Appendix A: Prompt for step 6

Open a new conversation with an AI assistant of your choice and give it exactly the following text, with no additional context and no follow-up clarification in the first round.

```
I have a Python class representing a symmetric Euclidean Traveling Salesman
Problem with the following interface:

class TSP:
  self.nCities # int, the number of cities
  self.cities # list of ints, 0 ..  nCities-1
  self.distMatrix # numpy 2D array, distMatrix[i,j] is the distance from
city i to city j
  def computeCosts(self, tour) # tour is a list of ints; returns the length
of the closed tour


Write a method makeTwoOpt_AI(self, tour) that applies 2-exchanges to the
given tour until no improving 2-exchange remains, and returns the resulting
tour.  Return only the method.
```